

Laborprojekt: Multimodale Fusion

Vorlesung EMI · Sommersemester 2026

Hochschule Mannheim · Institut A²IR

Worum geht es?

Sie haben in der Vorlesung mehrere Strategien kennengelernt, wie sich Information aus verschiedenen Modalitäten kombinieren lässt: Early Fusion, Late Fusion mit verschiedenen Aggregationsvarianten, Tensor Fusion, Low-Rank Fusion und Cross-Attention. In diesem Projekt setzen Sie zwei davon praktisch um und vergleichen sie auf einem realen Datensatz.

Das Ziel ist nicht, ein neues Modell zu erfinden, sondern saubere experimentelle Praxis zu üben: Daten vorbereiten, mehrere Modellvarianten implementieren, fair vergleichen, Ergebnisse interpretieren.

1 Aufgabenstellung

Implementieren Sie ein multimodales Klassifikationssystem mit **mindestens zwei** der in der Vorlesung behandelten Fusionsmethoden. Vergleichen Sie die Methoden experimentell und dokumentieren Sie Ihre Ergebnisse im Code-Repository.

1.1 Methodenwahl

Wählen Sie zwei aus der folgenden Liste. Mindestens eine Methode muss über reine Konkatenation hinausgehen.

- **Early Fusion** (Konkatenation, optional mit Projektion auf gemeinsame Dimension)
- **Late Fusion** mit statischer Gewichtung, Confidence-Gating oder lernbarem Gating
- **Cross-Attention** (uni- oder bidirektional)
- **Tensor Fusion (TFN)** oder **Low-Rank Fusion (LMF)**

1.2 Datensatzwahl

Sie haben freie Wahl. Drei Vorschläge zur Orientierung, von denen wir den ersten ausführlich dokumentieren:

- **RAVDESS** (Audio + Video, Emotionserkennung, kompakt, gut dokumentiert)
- **Hateful Memes** (Text + Bild, binäre Klassifikation, mittlere Grösse)
- **Memotion** (Bild + Text, Sentiment auf Memes, klein)

Andere Datensätze sind erlaubt, aber sprechen Sie das vorher kurz mit mir ab. Wichtig ist nur: mindestens zwei Modalitäten und saubere Labels.

1.3 Was Sie abgeben

Ein Git-Repository mit:

- Lauffähigem Code (Daten-Loader, Modelle, Training, Auswertung)
- Ausreichender Kommentierung: Docstrings auf Modellebene, Inline-Kommentare an kritischen Stellen
- Einer README mit Setup-Anleitung, Ergebnistabelle und kurzer Diskussion

2 Schritt-für-Schritt-Anleitung

Diese Anleitung verwendet **RAVDESS** als Beispiel. Für Hateful Memes oder andere Datensätze ist das Vorgehen analog, die Unterschiede stehen in Kapitel 3.

Die zentrale Strategie heisst **pre-extracted features**: Sie nutzen vortrainierte Modelle, um Ihre Audio- und Video-Daten einmalig in dichte Embeddings umzuwandeln. Diese Embeddings speichern Sie als NumPy-Arrays. Das eigentliche Fusion-Training läuft danach nur noch auf diesen Embeddings, nicht mehr auf den rohen Daten.

2.1 Was Sie nicht selbst trainieren müssen

Alle Encoder, die Sie brauchen, gibt es als vortrainierte Modelle. Sie laden sie aus HuggingFace und nutzen sie als Feature-Extraktoren. Keiner dieser Encoder wird während Ihres Projekts trainiert, alle bleiben eingefroren.

Modalität	Vortrainiertes Modell	Embedding-Dim
Audio (Speech)	facebook/wav2vec2-base	768
Audio (Speech, kleiner)	facebook/hubert-base-ls960	768
Video (Gesicht)	DeepFace oder OpenFace AU-Features	17 bis 512
Text	sentence-transformers/all-MiniLM-L6-v2	384
Bild	openai/clip-vit-base-patch32	512

2.2 Schritt 1: Setup

Installieren Sie die nötigen Pakete:

```
pip install torch torchaudio transformers
pip install librosa numpy pandas scikit-learn tqdm
pip install deepface # nur für Video
```

2.3 Schritt 2: Daten herunterladen

RAVDESS gibt es auf Zenodo, frei und ohne Anmeldung. Sie brauchen die Audio-Video-Datei pro Sprecher:

```
# https://zenodo.org/record/1188976
# Datei: Video_Speech_Actor_NN.zip für jeden Sprecher
# Audio extrahieren mit ffmpeg
ffmpeg -i video.mp4 -vn -acodec pcm_s16le -ar 16000 audio.wav
```

Die Dateinamen folgen einem Schema, aus dem Sie Sprecher, Emotion und weitere Metadaten direkt ablesen können (siehe RAVDESS-Dokumentation).

2.4 Schritt 3: Audio-Embeddings extrahieren

Einmalig durchziehen, dann auf Disk speichern. Auf CPU dauert das pro Datei wenige Sekunden, für den ganzen Datensatz unter einer Stunde.

```
import torch, torchaudio, numpy as np
from transformers import Wav2Vec2Model, Wav2Vec2FeatureExtractor

model_name = "facebook/wav2vec2-base"
extractor = Wav2Vec2FeatureExtractor.from_pretrained(model_name)
```

```

model = Wav2Vec2Model.from_pretrained(model_name).eval()

def audio_embedding(wav_path):
    waveform, sr = torchaudio.load(wav_path)
    if sr != 16000:
        waveform = torchaudio.functional.resample(waveform, sr, 16000)
    inputs = extractor(waveform.squeeze().numpy(),
                       sampling_rate=16000, return_tensors="pt")
    with torch.no_grad():
        out = model(**inputs).last_hidden_state # (1, T, 768)
    return out.mean(dim=1).squeeze().numpy()    # (768,)

```

Hinweis: `mean(dim=1)` aggregiert über die Zeitachse zu einem festen Vektor. Wenn Sie Cross-Attention auf der Audio-Sequenz machen wollen, behalten Sie stattdessen die Sequenz und poolen sie zum Beispiel auf 30 bis 50 Tokens herunter.

2.5 Schritt 4: Video-Embeddings extrahieren

DeepFace liefert Ihnen Emotion-Scores und Embeddings pro Frame. Schneller Workflow:

```

from deepface import DeepFace
import cv2, numpy as np

def video_embedding(video_path, n_frames=8):
    cap = cv2.VideoCapture(video_path)
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    idxs = np.linspace(0, total-1, n_frames, dtype=int)
    embeds = []
    for i in idxs:
        cap.set(cv2.CAP_PROP_POS_FRAMES, i)
        ret, frame = cap.read()
        if not ret: continue
        try:
            rep = DeepFace.represent(frame, model_name="Facenet",
                                    enforce_detection=False)
            embeds.append(rep[0]["embedding"])
        except Exception:
            continue
    cap.release()
    return np.mean(embeds, axis=0) if embeds else np.zeros(128)

```

Das ergibt einen 128-dimensionalen Gesichts-Embedding-Vektor pro Video. Alternativen: Action-Units mit OpenFace, oder rohe ViT-Features pro Frame.

2.6 Schritt 5: Embeddings speichern

Iterieren Sie einmalig über den ganzen Datensatz und speichern Sie alles in einer einzigen Datei:

```

import numpy as np
import pandas as pd

records = []
for path in all_files:
    h_a = audio_embedding(audio_path(path))
    h_v = video_embedding(video_path(path))
    label = parse_label(path)
    speaker = parse_speaker(path)
    records.append({"h_a": h_a, "h_v": h_v,
                  "label": label, "speaker": speaker})

```

```
np.savez("ravdess_features.npz",
        h_a=np.stack([r["h_a"] for r in records]),
        h_v=np.stack([r["h_v"] for r in records]),
        label=np.array([r["label"] for r in records]),
        speaker=np.array([r["speaker"] for r in records]))
```

Ab hier brauchen Sie die rohen Daten nicht mehr.

2.7 Schritt 6: PyTorch-Dataset und Splits

```
from torch.utils.data import Dataset, DataLoader
import torch, numpy as np

class RAVDESSFeatures(Dataset):
    def __init__(self, npz_path, indices):
        data = np.load(npz_path)
        self.h_a = data["h_a"][indices]
        self.h_v = data["h_v"][indices]
        self.y = data["label"][indices]
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        return (torch.from_numpy(self.h_a[i]).float(),
                torch.from_numpy(self.h_v[i]).float(),
                int(self.y[i]))
```

Wichtig: Splits sprecher-unabhängig machen. Wenn derselbe Sprecher in Train und Test landet, überschätzen Sie die Generalisierung dramatisch.

```
from sklearn.model_selection import GroupShuffleSplit

data = np.load("ravdess_features.npz")
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(data["h_a"], data["label"],
                                     groups=data["speaker"]))
```

2.8 Schritt 7: Modelle implementieren

Hier kommt der Kern Ihres Projekts. Beispiele für zwei Methoden:

Early Fusion.

```
import torch.nn as nn

class EarlyFusion(nn.Module):
    def __init__(self, d_a=768, d_v=128, n_classes=8, hidden=128):
        super().__init__()
        self.norm_a = nn.LayerNorm(d_a)
        self.norm_v = nn.LayerNorm(d_v)
        self.head = nn.Sequential(
            nn.Linear(d_a + d_v, hidden),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(hidden, n_classes),
        )
    def forward(self, h_a, h_v):
        z = torch.cat([self.norm_a(h_a), self.norm_v(h_v)], dim=-1)
```

```
return self.head(z)
```

Late Fusion mit lernbarem Gating.

```
class LateFusionGated(nn.Module):
    def __init__(self, d_a=768, d_v=128, n_classes=8, hidden=64):
        super().__init__()
        self.c_a = nn.Linear(d_a, n_classes)
        self.c_v = nn.Linear(d_v, n_classes)
        self.gate = nn.Sequential(
            nn.Linear(d_a + d_v, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 2),
        )
    def forward(self, h_a, h_v):
        p_a = torch.softmax(self.c_a(h_a), dim=-1)
        p_v = torch.softmax(self.c_v(h_v), dim=-1)
        w = torch.softmax(self.gate(torch.cat([h_a, h_v], dim=-1)), dim=-1)
        return torch.log(w[:, 0:1] * p_a + w[:, 1:2] * p_v + 1e-8)
```

Beachten Sie das log am Schluss: Die Mischung erfolgt im Wahrscheinlichkeitsraum, der CrossEntropyLoss erwartet aber Logits. Alternativ NLLLoss verwenden.

2.9 Schritt 8: Training

```
def train_model(model, loader_train, loader_val, n_epochs=30, lr=1e-3):
    opt = torch.optim.AdamW(model.parameters(), lr=lr)
    crit = nn.CrossEntropyLoss()
    best_f1, best_state = 0.0, None
    for epoch in range(n_epochs):
        model.train()
        for h_a, h_v, y in loader_train:
            opt.zero_grad()
            logits = model(h_a, h_v)
            loss = crit(logits, y)
            loss.backward()
            opt.step()
        f1 = evaluate_macro_f1(model, loader_val)
        if f1 > best_f1:
            best_f1, best_state = f1, model.state_dict()
    model.load_state_dict(best_state)
    return model, best_f1
```

Mit pre-extracted Features läuft eine Epoche auf RAVDESS in unter einer Minute auf CPU. 30 Epochen also rund eine halbe Stunde.

2.10 Schritt 9: Auswertung

Drei Seeds pro Methode laufen lassen, Mittelwert und Standardabweichung berichten.

```
from sklearn.metrics import f1_score, confusion_matrix

def evaluate_macro_f1(model, loader):
    model.eval()
    y_true, y_pred = [], []
    with torch.no_grad():
        for h_a, h_v, y in loader:
```

```

        logits = model(h_a, h_v)
        y_pred.extend(logits.argmax(dim=-1).tolist())
        y_true.extend(y.tolist())
    return f1_score(y_true, y_pred, average="macro")

```

Pflicht für den Bericht:

- Tabelle: Methode mal Metrik (Macro-F1, Accuracy), mit Mittelwert und Standardabweichung über drei Seeds
- Unimodale Baselines pro Modalität als Referenzpunkt
- Konfusionsmatrix für die beste Methode
- Modality-Ablation: Was passiert, wenn Sie zur Inferenz eine Modalität auf null setzen?
- Drei bis fünf Sätze Diskussion: welche Methode war besser und warum

3 Variante: Hateful Memes (Text + Bild)

Wenn Sie statt Audio-Video lieber mit Text und Bild arbeiten, wechseln Sie die Encoder aus. Das Schema bleibt identisch.

3.1 Daten

Hateful Memes von Facebook AI, verfügbar über DrivenData. Etwa 10.000 Memes mit binärem Label (hasserfüllt oder nicht). Registrierung nötig, dafür sehr saubere Aufbereitung.

3.2 Encoder

Text-Embeddings.

```

from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
def text_embedding(caption):
    return model.encode(caption)  # (384,)

```

Bild-Embeddings via CLIP.

```

from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import torch

clip = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").eval()
proc = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

def image_embedding(img_path):
    img = Image.open(img_path).convert("RGB")
    inputs = proc(images=img, return_tensors="pt")
    with torch.no_grad():
        feat = clip.get_image_features(**inputs)  # (1, 512)
    return feat.squeeze().numpy()

```

Ab hier ist alles wie in den Schritten 5 bis 9. Sie ersetzen nur `h_a` und `h_v` durch `h_t` (Text) und `h_i` (Bild). Die Modellklassen funktionieren mit anderen Dimensionen, Sie müssen nur `d_a` und `d_v` entsprechend setzen.

3.3 Tipp

CLIP ist von Haus aus auf Bild-Text-Alignment trainiert. Eine extrem starke Baseline auf Hateful Memes ist daher: Cosinus-Ähnlichkeit zwischen CLIP-Text- und CLIP-Bild-Embedding, plus ein lineares Modell drauf. Wenn Ihre Fusion-Methoden diese Baseline nicht schlagen, ist das interessant und gehört in die Diskussion.

4 Hardware-Hinweise

Das gesamte Projekt ist so ausgelegt, dass es auf einem normalen Laptop läuft. Konkret:

- **Feature-Extraktion** (einmalig): RAVDESS in unter einer Stunde auf CPU. Hateful Memes etwas länger wegen Datensatzgrösse.
- **Training** der Fusion-Modelle: 30 Epochen pro Seed in wenigen Minuten auf CPU.
- **Gesamtaufwand**: ein bis zwei Tage realistisch, inklusive Berichterstellung.

GPU nicht erforderlich. Wenn Sie eine haben (oder Google Colab nutzen), wird die Feature-Extraktion schneller, das Training selbst macht kaum einen Unterschied.

5 Bewertung

Bereich	Punkte	Was wir bewerten
Datenpipeline und Splits	15	Sprecher-unabhängige Splits, Reproduzierbarkeit
Implementierung Methode 1	25	Korrektheit, sinnvolle Hyperparameter, lauffähig
Implementierung Methode 2	25	wie oben
Experimente und Auswertung	15	Drei Seeds, Ablation, Konfusionsmatrix
Code-Qualität und Kommentierung	15	Lesbar, dokumentiert, reproduzierbar
Diskussion in der README	5	Plausible Interpretation der Ergebnisse
Bonus	bis 10	Dritte Methode, eigener Datensatz, Attention-Visualisierung

6 Häufige Stolpersteine

- **Splits auf Sample-Ebene statt Sprecher-Ebene.** Führt zu unrealistisch guten Ergebnissen. Immer mit `GroupShuffleSplit` oder `LeaveOneGroupOut`.
- **Fehlende LayerNorm bei Early Fusion.** Audio-Embeddings sind oft betragsmässig viel grösser als Video-Embeddings. Ohne Normalisierung dominiert eine Modalität.
- **Nur eine Seed.** Ergebnisse sind nicht aussagekräftig. Drei Seeds sind das Minimum.
- **Vergessene unimodale Baselines.** Ohne diese können Sie nicht zeigen, dass die Fusion überhaupt etwas bringt.
- **Encoder trainieren wollen.** Die Encoder sind eingefroren, sonst läuft das nicht auf CPU.

7 Abgabe

Link zum Git-Repository per Mail. Im Repository:

- `README.md` mit Setup-Anleitung, Ergebnistabelle und Diskussion
- `extract_features.py` und `train.py` (oder analog strukturiert)
- `results/` mit CSVs und Plots
- `requirements.txt`

Rohe Daten bitte nicht einchecken, nur die Anleitung zum Download.

Fragen? Sprechstunde, Forum, oder per Mail an mich. Vor allem bei der Datensatzwahl ist eine kurze Rückfrage sinnvoll, bevor Sie viel Zeit investieren.